

ZeroTrust Engine - Project Documentation

ZeroTrust Engine: Student Project Documentation

1. Project Abstract

Traditional network security relies on a “castle-and-moat” approach, where anyone inside the network is trusted. This project implements a **Zero Trust Architecture (ZTA)**, operating on the principle of “**Never Trust, Always Verify.**”

The **ZeroTrust Engine** is a modern web application that evaluates every access request dynamically. It calculates a risk score based on the user’s role, device compliance, location, and the sensitivity of the requested application, granting or denying access in real-time.

2. Technology Stack

- **Frontend:** React 18, TypeScript, Vite
 - **Styling:** Tailwind CSS, Lucide React (Icons)
 - **Backend / Database:** Supabase (PostgreSQL)
 - **State Management:** React Context API
 - **Charting/Visualization:** Recharts
 - **PDF Generation:** jsPDF (for compliance reporting)
-

3. System Architecture

The system consists of a Client (React UI), an Authentication/Context Layer, a Risk Evaluation Engine (Core Logic), and a Backend Database (Supabase).

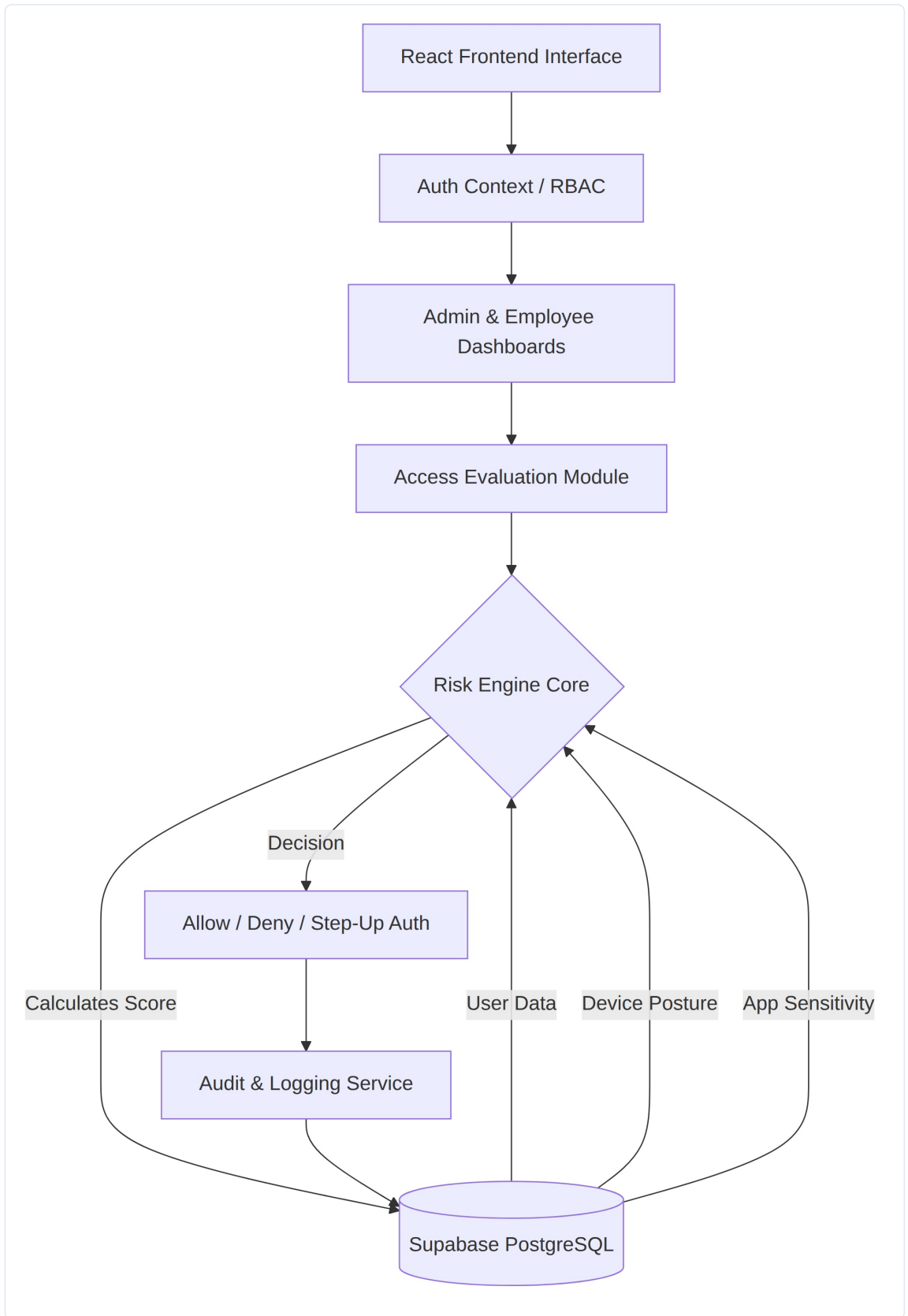


Diagram 1

4. Core Workflows (How it works)

4.1 Access Request Workflow (Sequence Diagram)

When a user attempts to access an application, the system evaluates their context before establishing a connection.

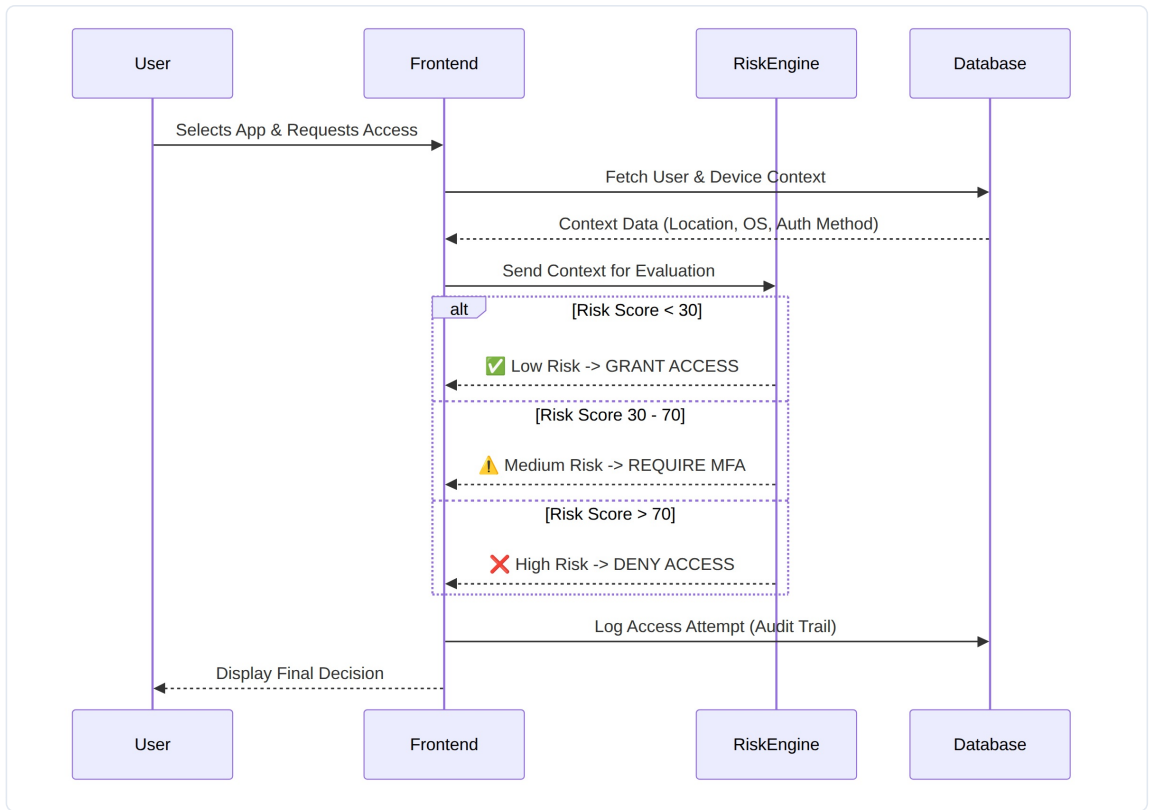
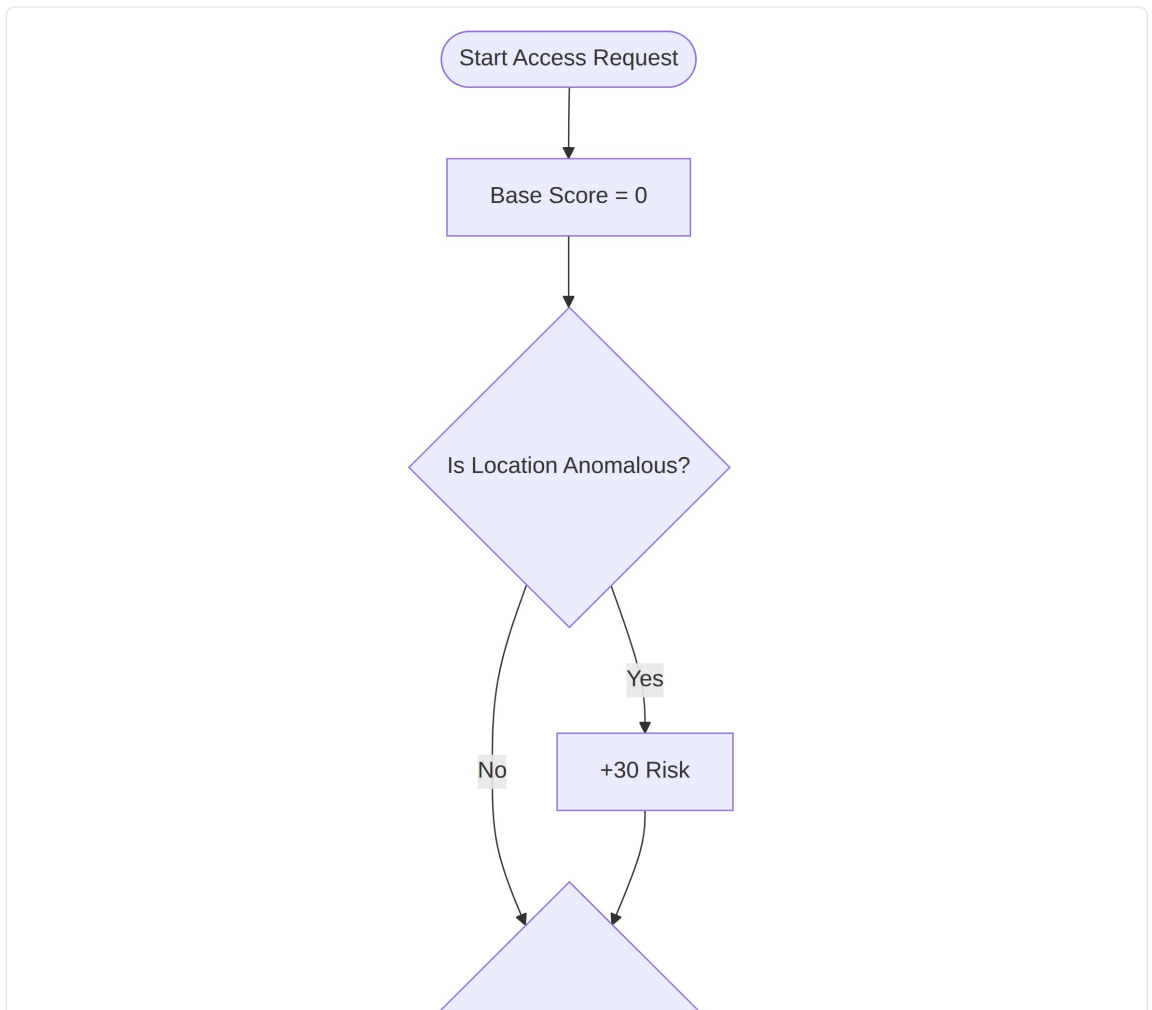


Diagram 2

4.2 Risk Scoring Logic (Flowchart)

This flowchart shows exactly how the `riskEngine.ts` file calculates the final threat level.



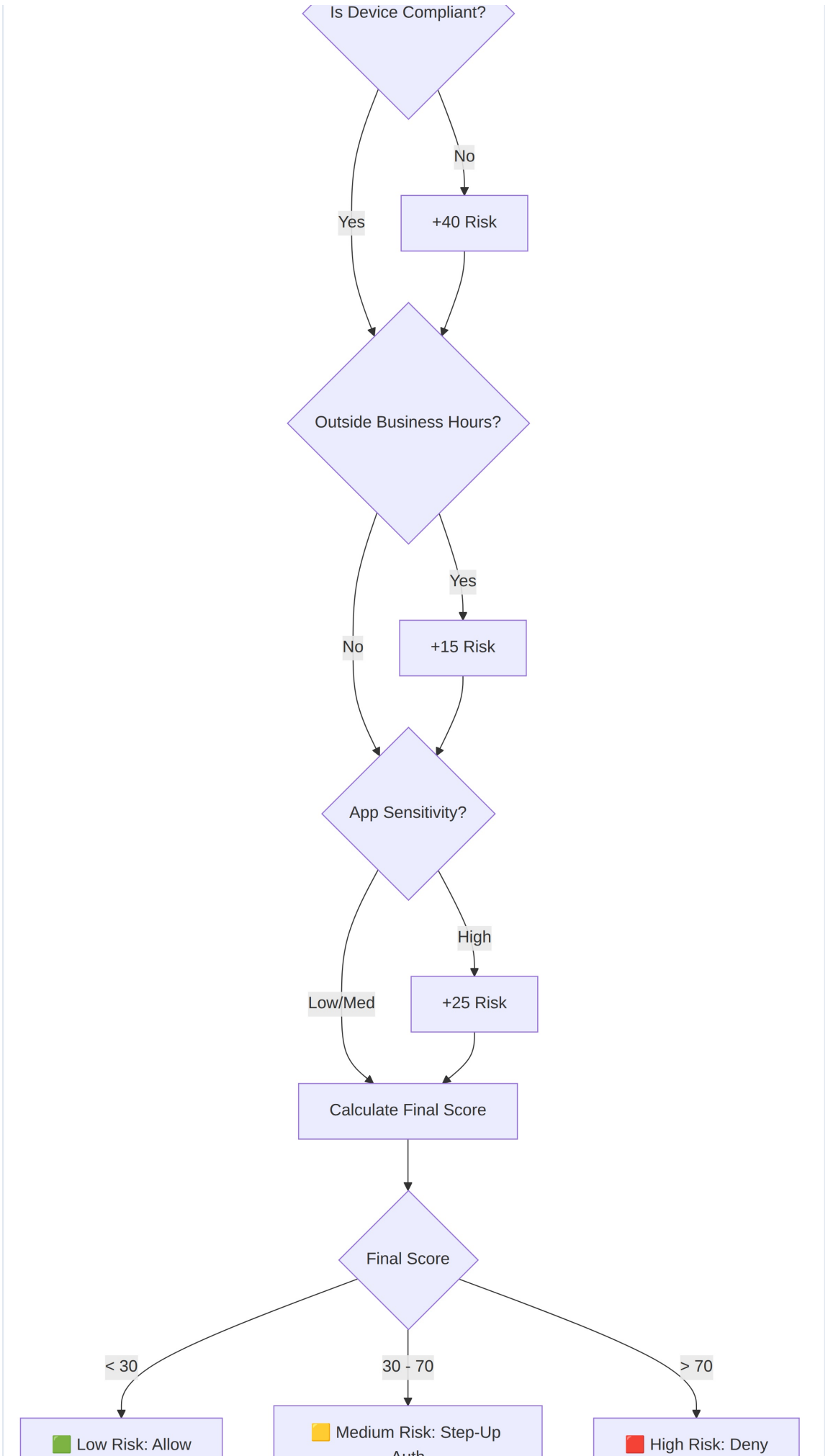




Diagram 3

5. Database Design (Entity Relationship / UML)

The database schema is highly relational, connecting users to devices, logging access attempts, and mapping network segments.

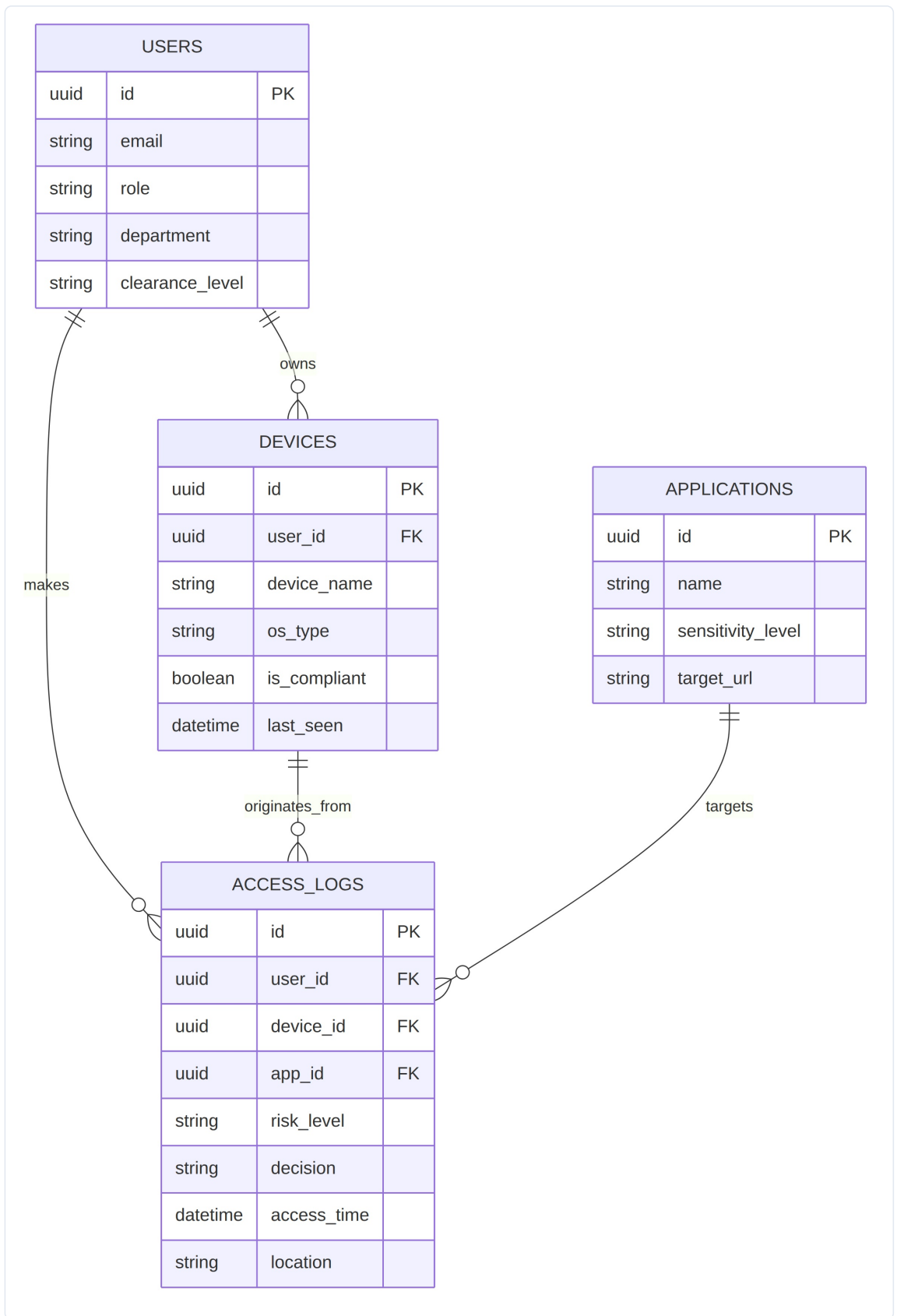


Diagram 4

6. Detailed Component Breakdown

A. Role-Based Access Control (RBAC)

Implemented in `AuthContext.tsx` and `Layout.tsx`. * **Admin:** Has full access to CRUD operations (Create, Read,

Update, Delete) for Users, Devices, and Applications. Can view the mathematical AI Risk Model. * **Employee:** Has restricted access. Can view their own dashboard, evaluate their own access requests, and view basic logs.

B. The Risk Engine (`src/lib/riskEngine.ts`)

A deterministic rules-engine that acts as the “Brain” of the Zero Trust model. It accepts an `EvaluationContext` object containing: * `userClearance` : (Standard, Confidential, Top Secret) * `deviceCompliant` : (True/False based on antivirus/OS updates) * `location` : (Known vs. Anomalous IP/Geolocations) * `time` : (Core business hours vs. unusual hours) * `mfaEnabled` : (Multi-factor authentication presence)

C. Glassmorphism UI & Video Background

The `LoginPage.tsx` implements modern web design principles including: * **Z-Index Layering:** Video at `z-0`, gradient overlays at `z-10`, and frosted glass cards at `z-20`. * **Backdrop Filters:** Uses `backdrop-filter: blur(30px)` to simulate frosted glass.

7. How to Explain This Project in a College Presentation

1. **The Problem:** “In older networks, once a hacker gets past the firewall, they have access to everything. This is dangerous.”
 2. **The Solution (Our Project):** “We built a Zero Trust system. Even if you have the right password, our engine checks your device, your location, and what time it is before letting you open an app.”
 3. **The Innovation:** “Instead of static rules, we calculate a dynamic **Risk Score (0-100)** on the fly. If the risk gets too high, we block the connection immediately.”
 4. **The Tech:** “We used React for a responsive dashboard and Supabase to log every single access attempt for security auditing.”
-

8. Local Setup & Execution Guide

To run the project on any machine: 1. Ensure **Node.js** is installed. 2. Open the terminal in the project folder. 3. Run `npm install` (to install dependencies). 4. Run `npm run dev` (to start the server). 5. Open the browser to the provided localhost URL (usually `http://localhost:5173`). 6. (Alternatively, just double-click the `run_project.bat` file in the folder!)